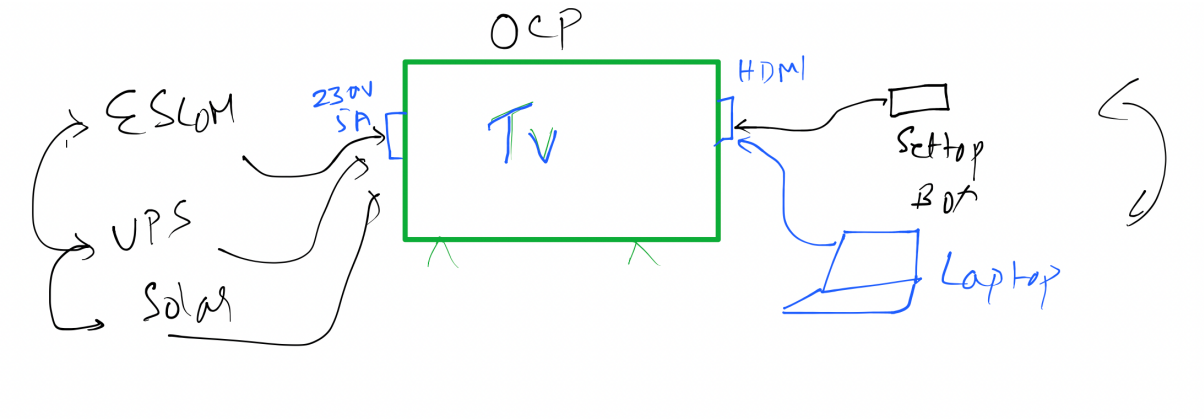
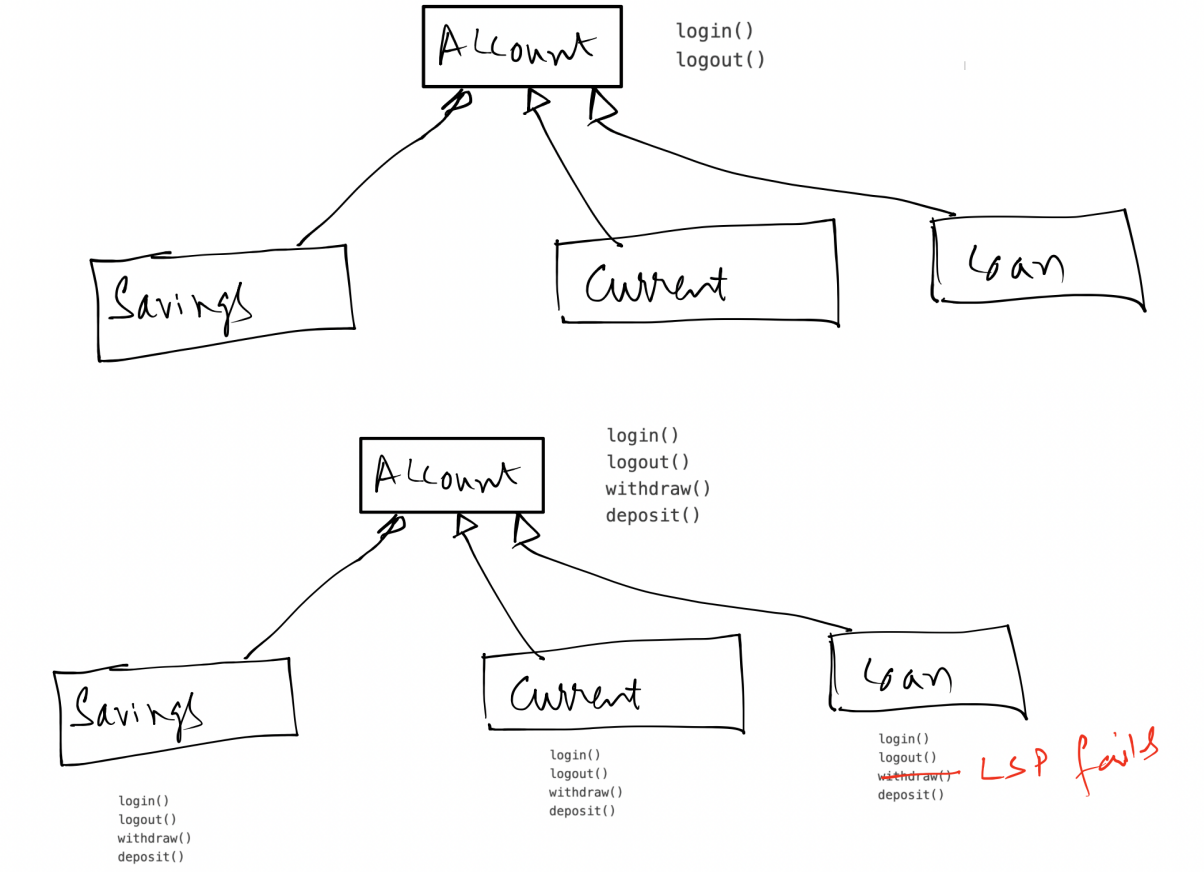
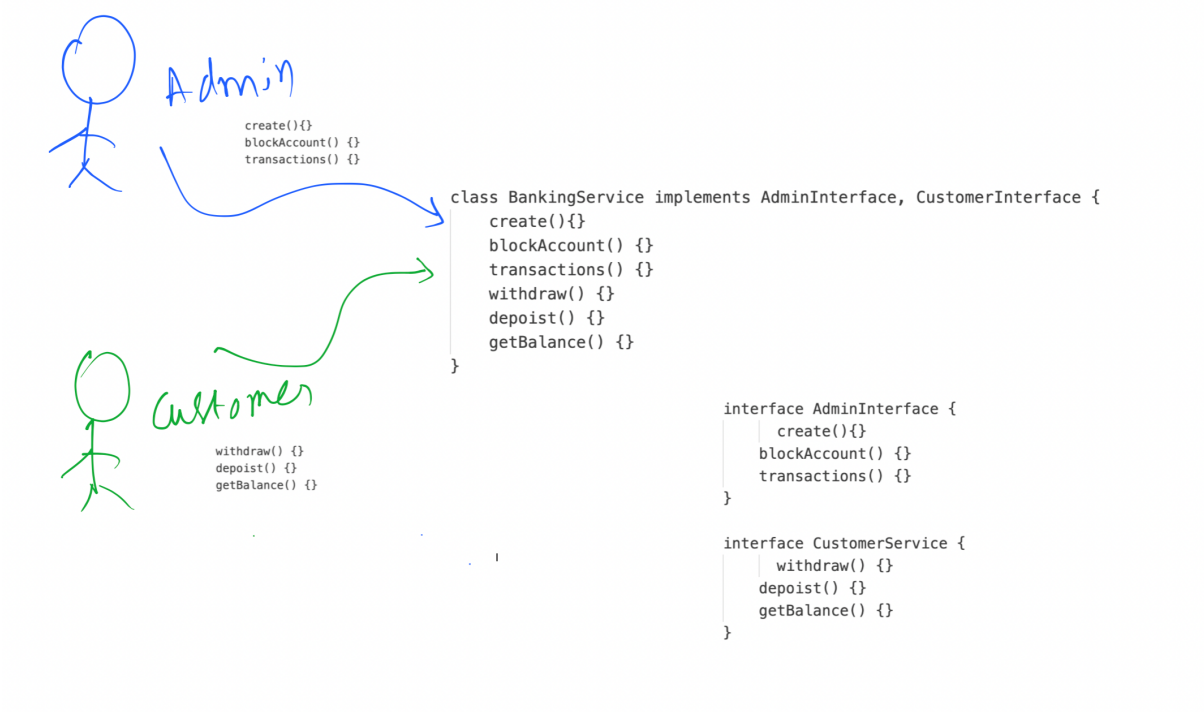
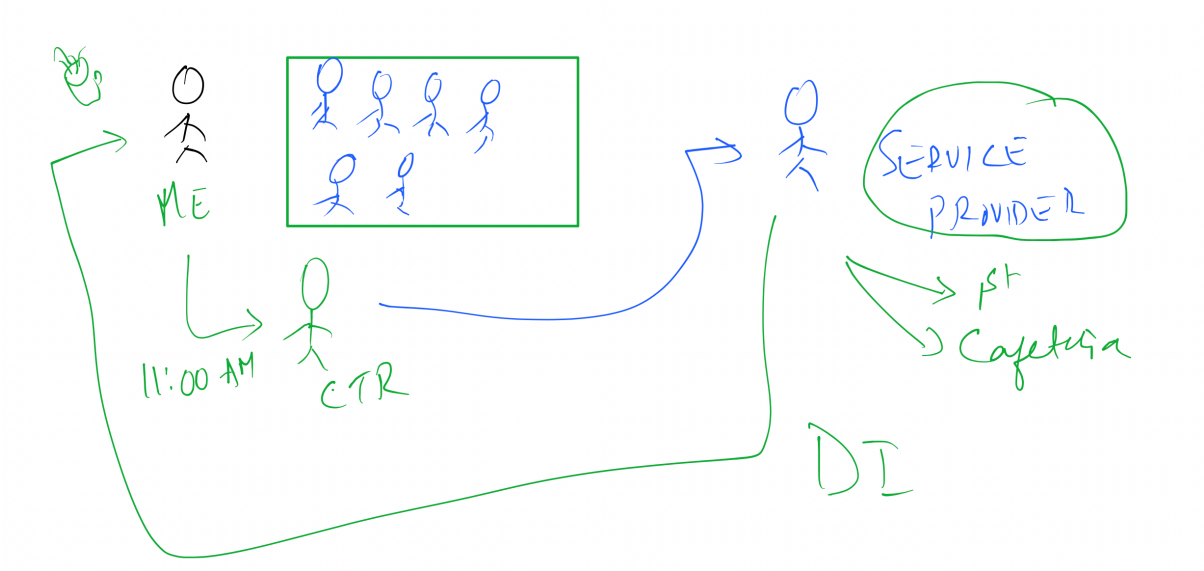

OCP
ESLOM
230V
SP
UPS
Solar
TV
HDMI
Settop
Box
Laptop

LSP:


Account
login()
logout()
Savings
Current
Loan
Account
login()
logout()
withdraw()
deposit()
Savings
Current
Loan
login()
logout()
withdraw()
deposit()
login()
logout()
withdraw()
deposit()
login()
logout()
withdraw()
deposit()
LSP fails

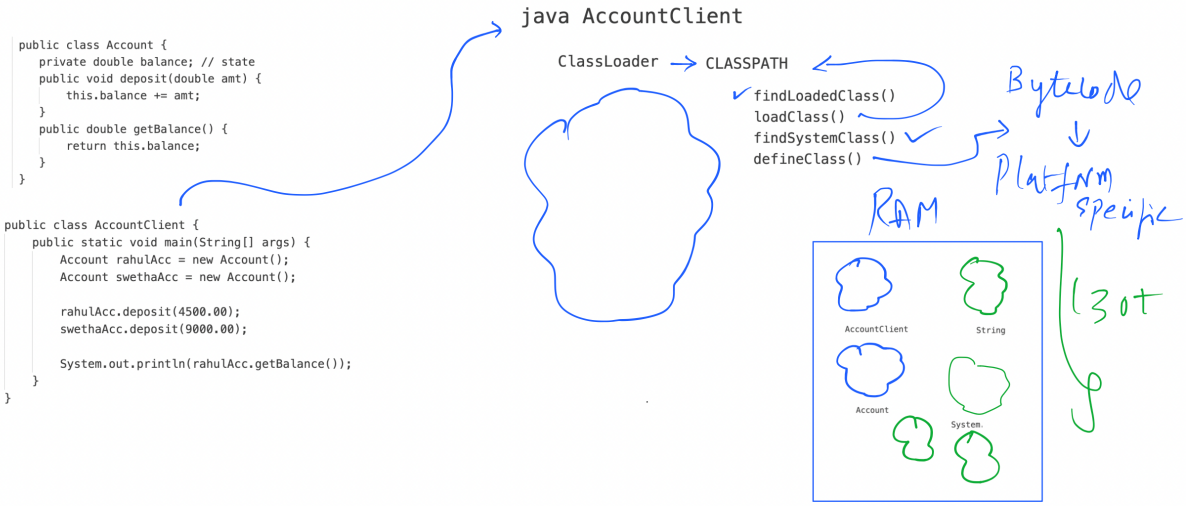


Dependency Injection:



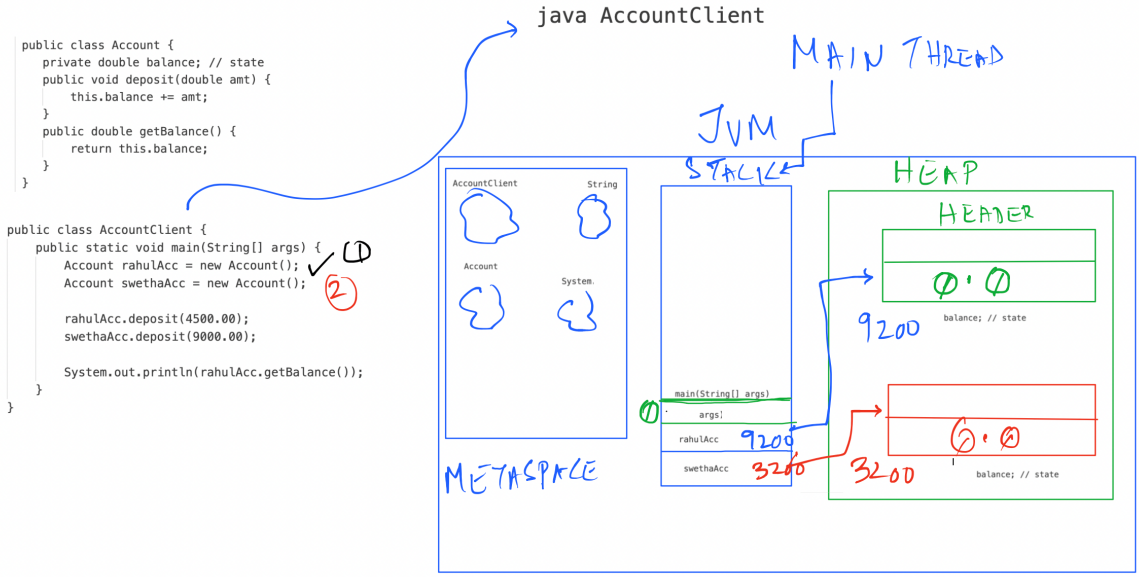
```
public class Account {
    private double balance; // state
    public void deposit(double amt) {
        this.balance += amt;
    }
    public double getBalance() {
        return this.balance;
    }
}
```

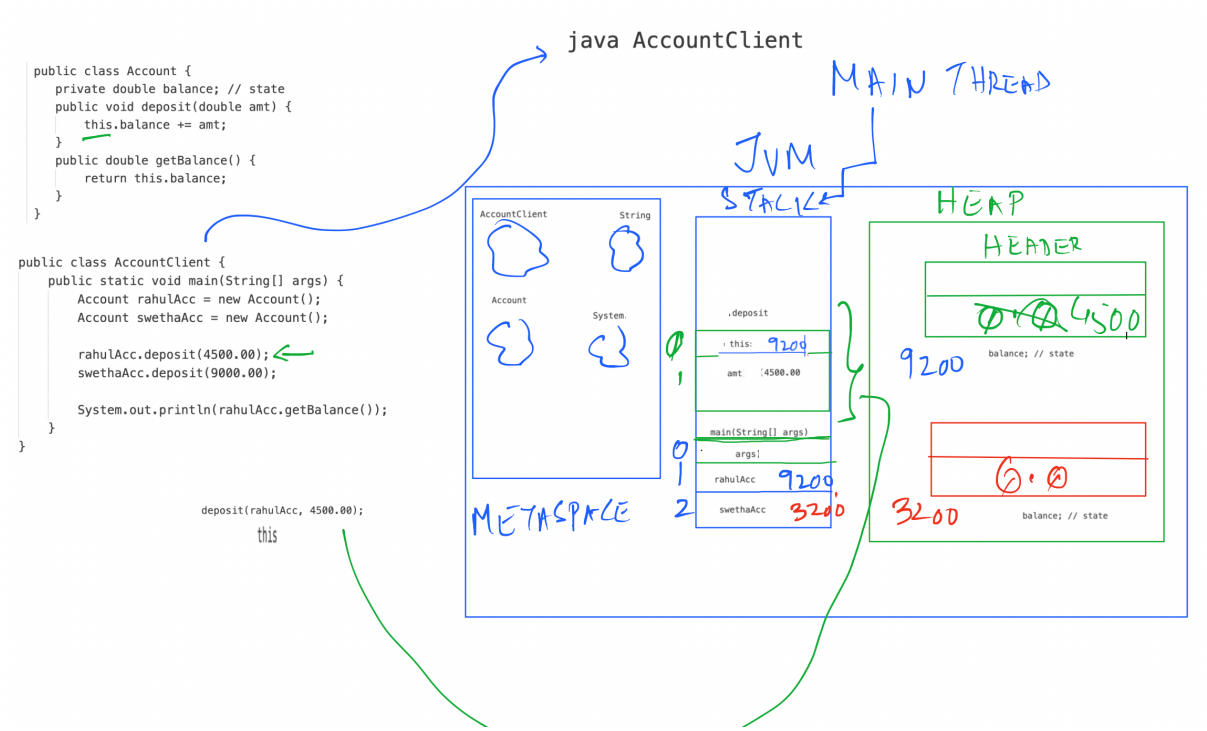
```
public class AccountClient {
    public static void main(String[] args) {
        Account rahulAcc = new Account();
        Account swethaAcc = new Account();

        rahulAcc.deposit(4500.00);
        swethaAcc.deposit(9000.00);

        System.out.println(rahulAcc.getBalance());
    }
}
```

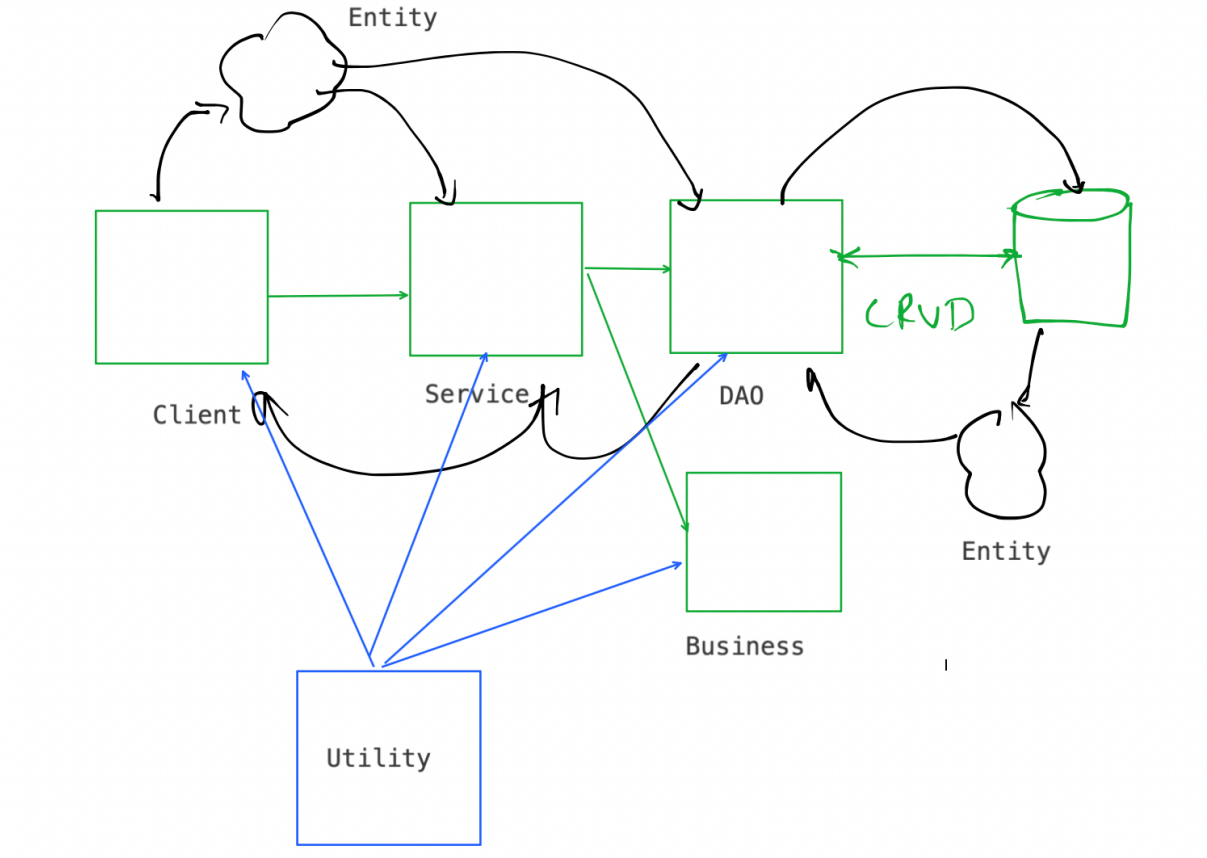



```
public class Account {
    private double balance; // state
    public void deposit(double amt) {
        this.balance += amt;
    }
    public double getBalance() {
        return this.balance;
    }
}
```

```
public class AccountClient {
    public static void main(String[] args) {
        Account rahulAcc = new Account();
        Account swethaAcc = new Account();

        rahulAcc.deposit(4500.00);
        swethaAcc.deposit(9000.00);

        System.out.println(rahulAcc.getBalance());
    }
}
```



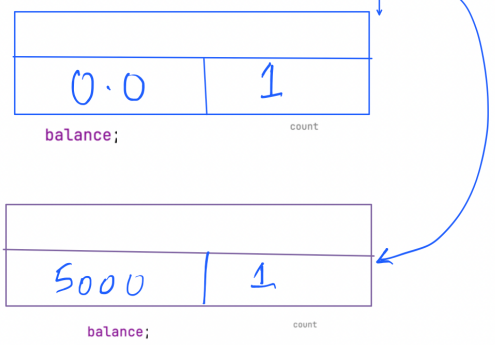

java AccountClient
MAIN THREAD
JVM
STACK
HEAP
HEADER
AccountClient
String
Account
System
.deposit
this: 9200
amt: 4500.00
main(String[] args)
args
rahulAcc 9200
swethaAcc 3200
4500
9200
balance; // state
0.0
3200
balance; // state
METASPACE
deposit(rahulAcc, 4500.00);
this

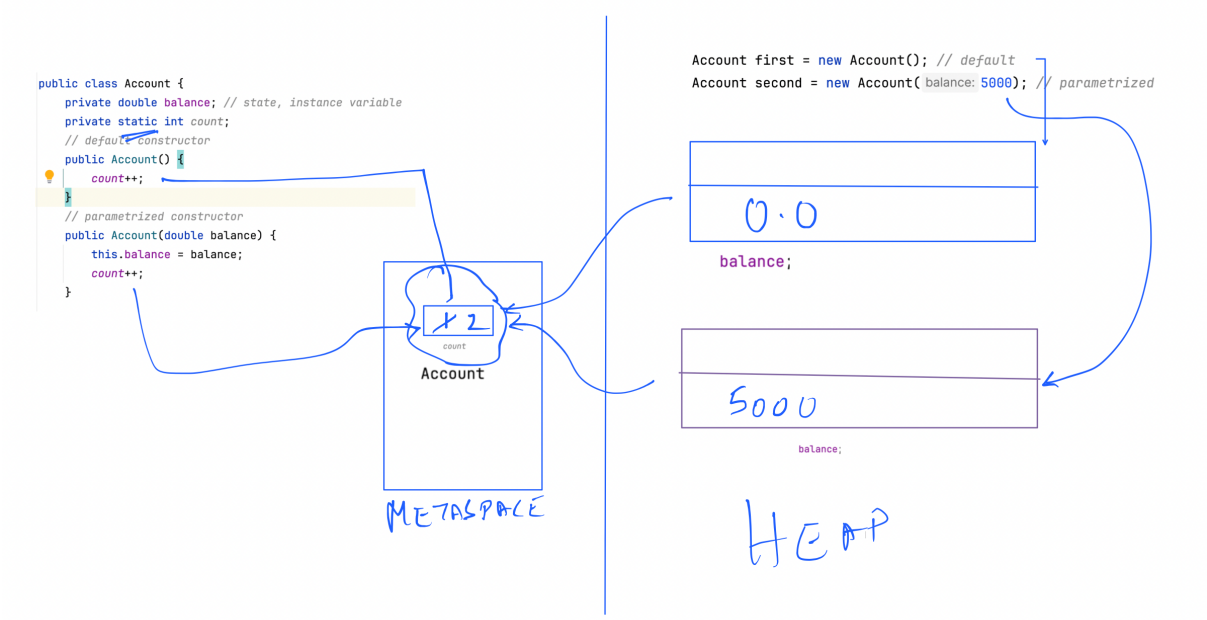

Entity
CRUD
Client
Service
DAO
Entity
Business
Utility

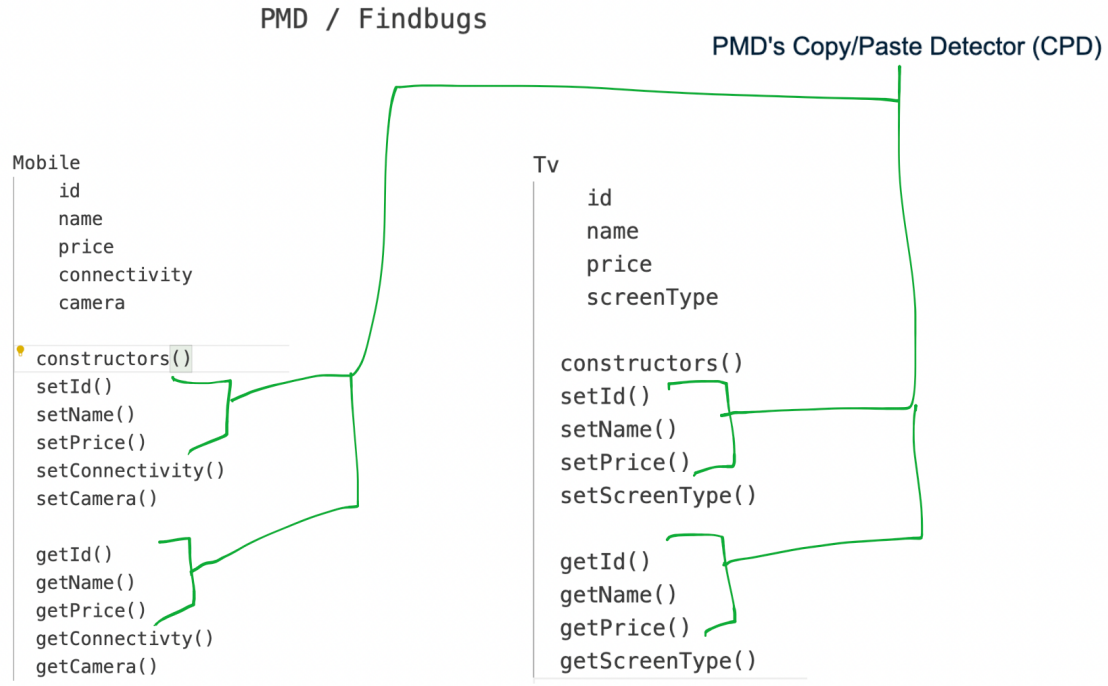
```
com
  |
  adobe
    |
    aem (prj/module)
      |
      entity
        |
        Customer.class [ mapped to Customer table / document]
        Product.class
        Order.class
      repo [per table]
        |
        CustomerDao.class [ CRUD for Customer table]
        ProductDao.class [ Crud for Product  table]
        OrderDao.class
      service
        |
        AdminService.class [for Actor Admin]
        CustomerService.class [ for Customer actor]
```

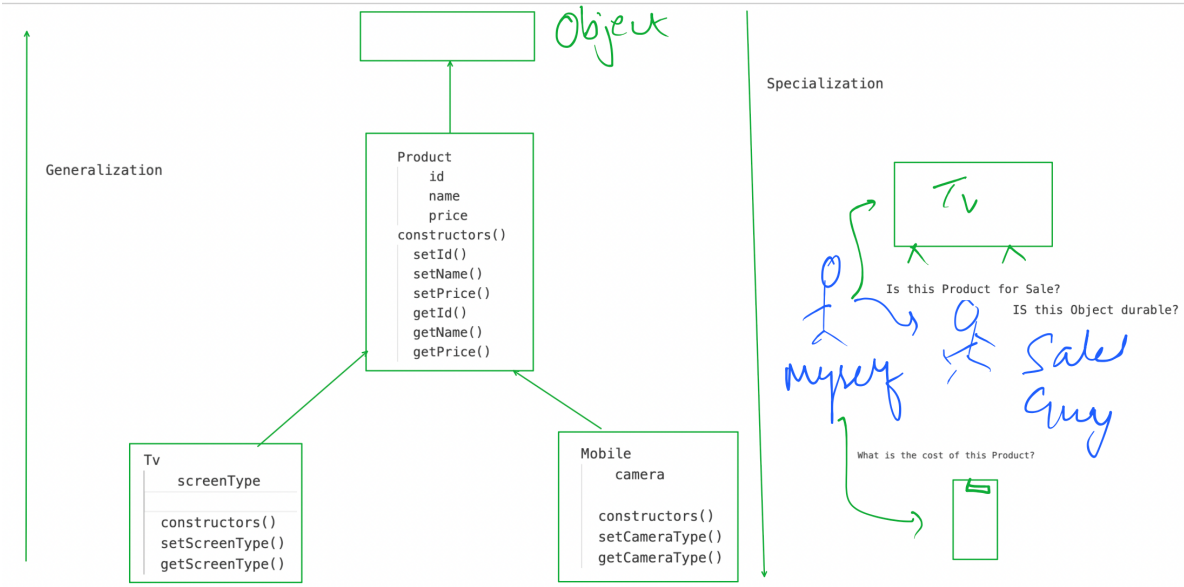
```java
public class Account {
    private double balance; // state, instance variable
    private int count;
    // default constructor
    public Account() {
        count++;
    }
    // parametrized constructor
    public Account(double balance) {
        this.balance = balance;
        count++;
    }
```

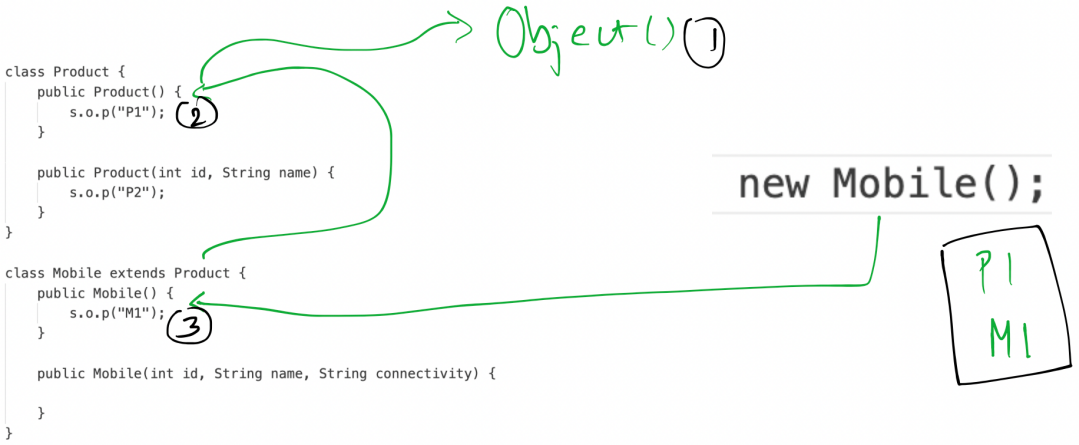
```java
Account first = new Account(); // default
Account second = new Account( balance: 5000); // parametrized
```

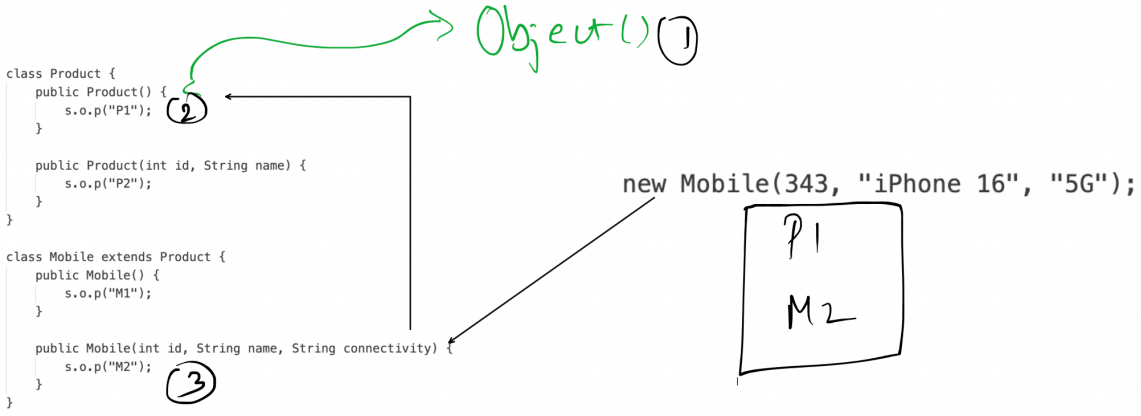



public class Account {
private double balance; // state, instance variable
private static int count;
// default constructor
public Account() {
count++;
}
// parametrized constructor
public Account(double balance) {
this.balance = balance;
count++;
}
Account first = new Account(); // default
Account second = new Account(balance: 5000); // parametrized
0.0
balance;
5000
balance;
count
Account
METASPACE
HEAP

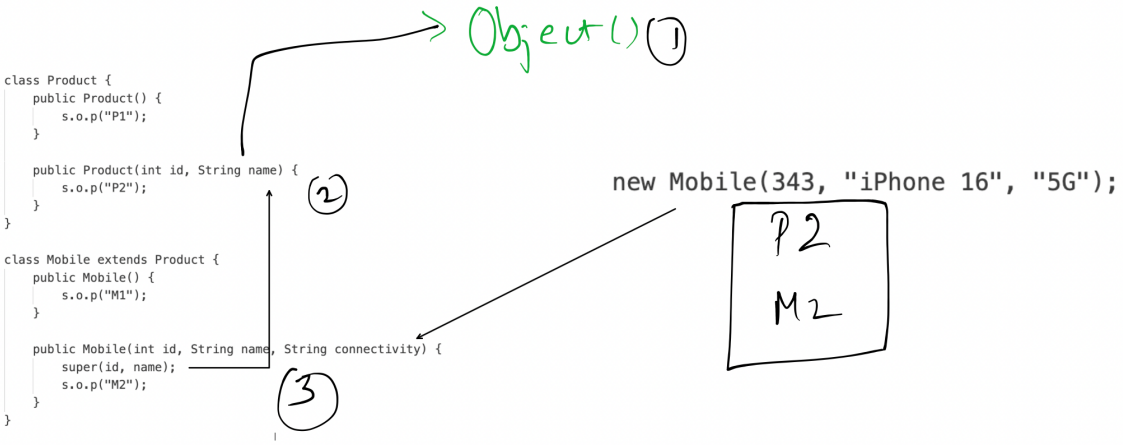

PMD / Findbugs
PMD's Copy/Paste Detector (CPD)
Mobile
id
name
price
connectivity
camera
constructors()
setId()
setName()
setPrice()
setConnectivity()
setCamera()
getId()
getName()
getPrice()
getConnectivty()
getCamera()
Tv
id
name
price
screenType
constructors()
setId()
setName()
setPrice()
setScreenType()
getId()
getName()
getPrice()
getScreenType()



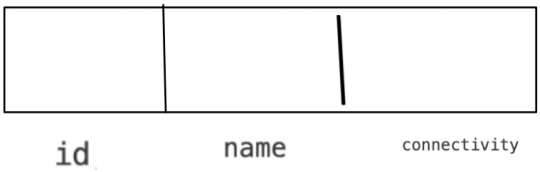


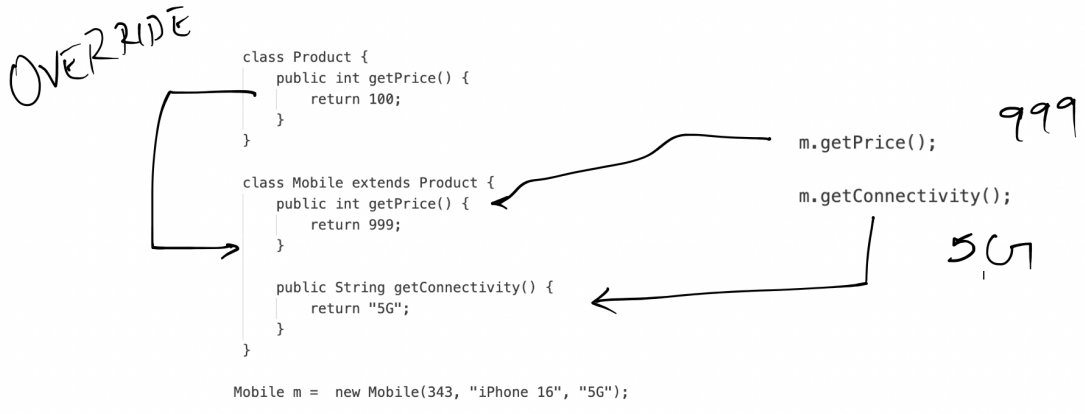


super() has to be the first statement




```
class Product {
    int id;
    String name;
}

class Mobile extends Product {
   String connectivity;
}

new Mobile(343, "iPhone 16", "5G");
```





```
class Product {
    public int getPrice() {
        return 100;
    }
}

class Mobile extends Product {
    public int getPrice() {
        return 999;
    }

    public String getConnectivity() {
        return "5G";
    }
}

Mobile m =  new Mobile(343, "iPhone 16", "5G");
```

m.getPrice(); 100

m.getConnectivity();

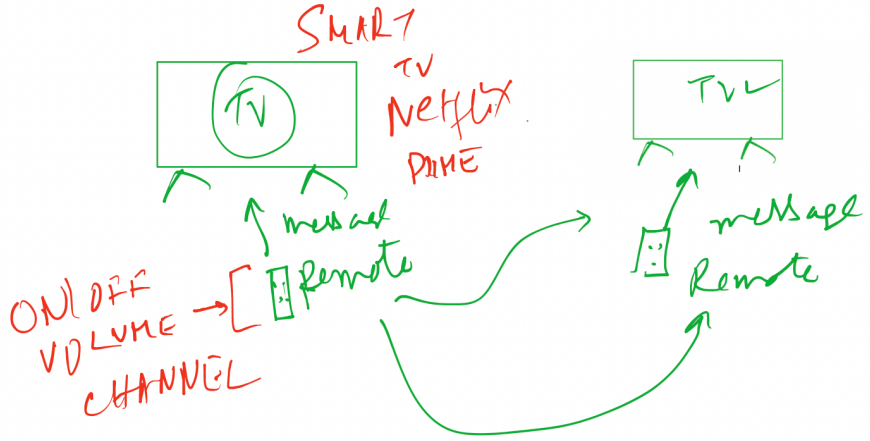
5G

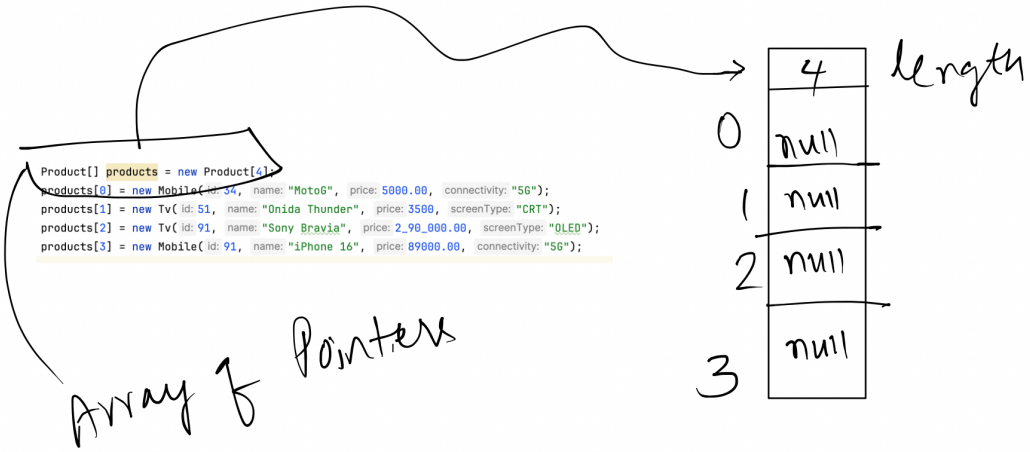
```
class Product {
    public int getPrice() {
        return 100;
    }
}

class Mobile extends Product {
    public int getPrice() {
        return 999;
    }

    public String getConnectivity() {
        return "5G";
    }
}
```

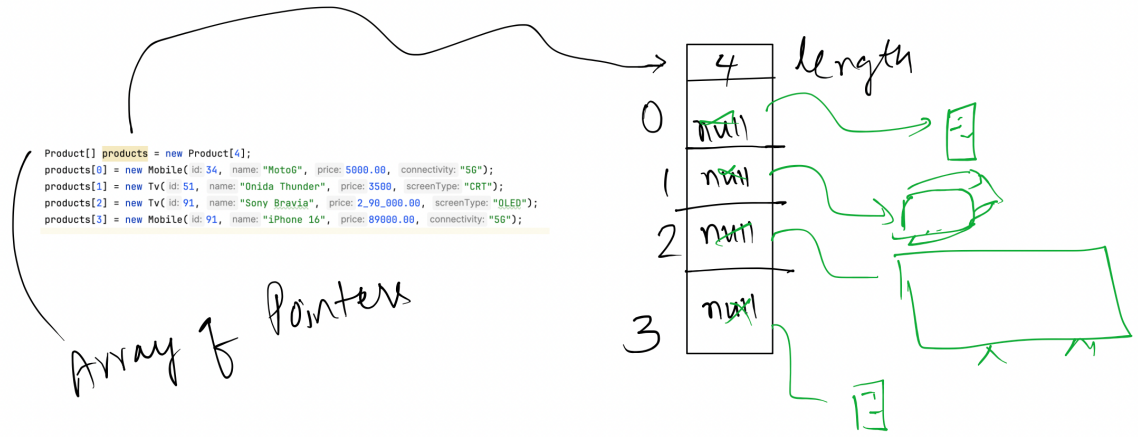
Dynamic Binding

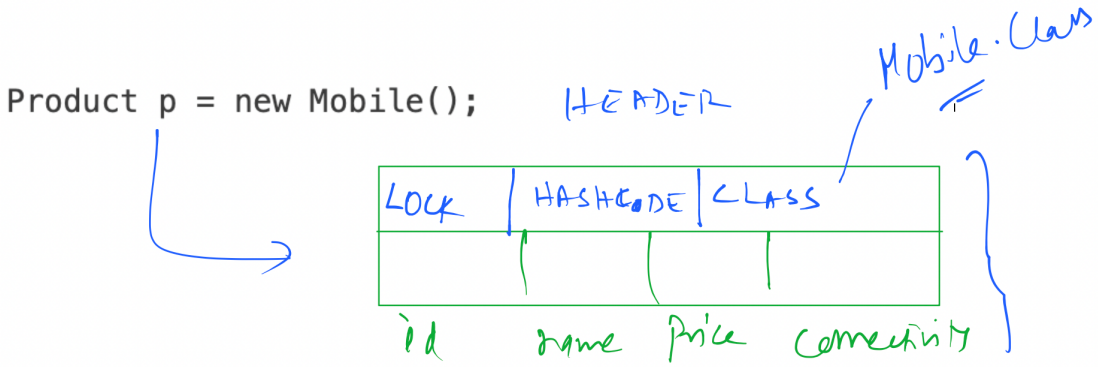
```
Product p =  new Mobile();
p.getPrice();
p.getConnectivity();
```

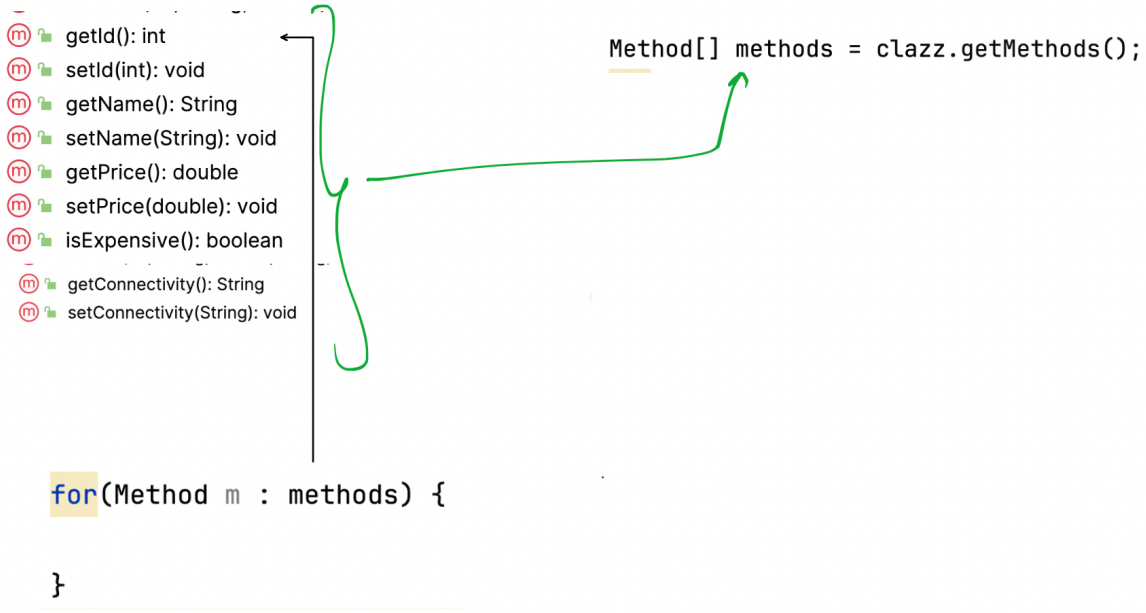
999

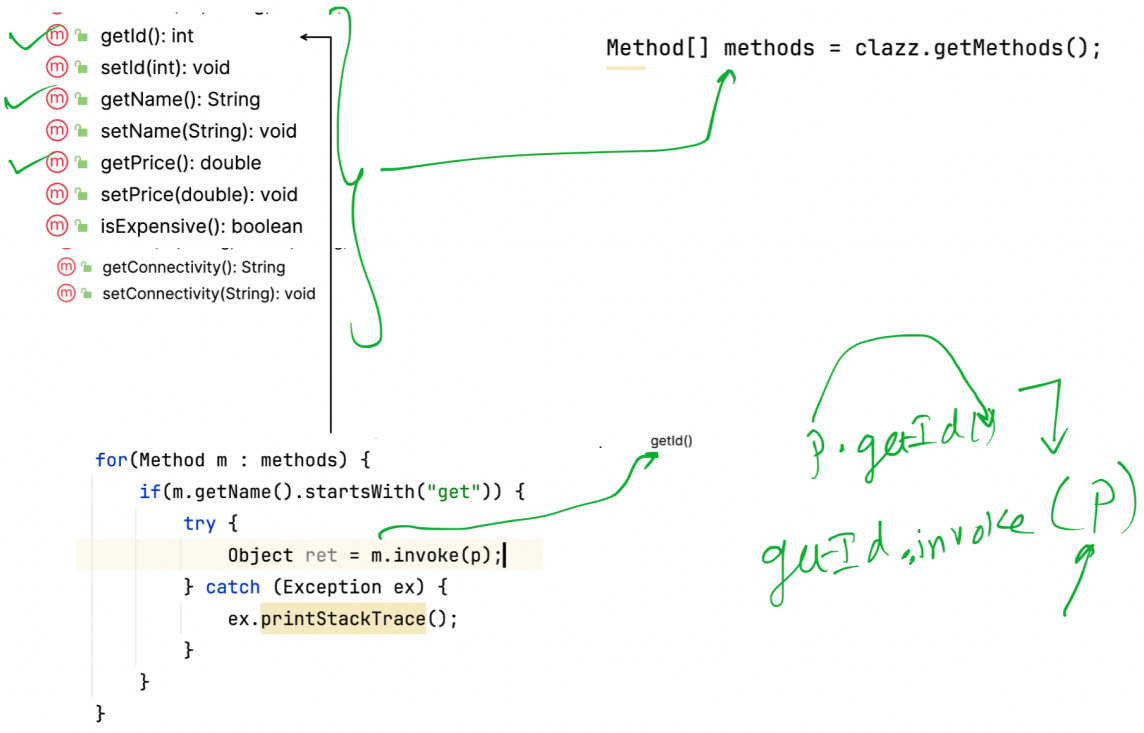



Product[] products = new Product[4];
products[0] = new Mobile(id: 34, name: "MotoG", price: 5000.00, connectivity: "5G");
products[1] = new Tv(id: 51, name: "Onida Thunder", price: 3500, screenType: "CRT");
products[2] = new Tv(id: 91, name: "Sony Bravia", price: 2_90_000.00, screenType: "OLED");
products[3] = new Mobile(id: 91, name: "iPhone 16", price: 89000.00, connectivity: "5G");
Array of Pointers
4
length
0
null
1
null
2
null
3
null


Product[] products = new Product[4];
products[0] = new Mobile(id: 34, name: "MotoG", price: 5000.00, connectivity: "5G");
products[1] = new Tv(id: 51, name: "Onida Thunder", price: 3500, screenType: "CRT");
products[2] = new Tv(id: 91, name: "Sony Bravia", price: 2_90_000.00, screenType: "OLED");
products[3] = new Mobile(id: 91, name: "iPhone 16", price: 89000.00, connectivity: "5G");
Array of Pointers
4
length
0
1
2
3


Product p = new Mobile();
HEADER
Mobile.class
LOCK
HASHCODE
CLASS
id
name
price
connectivity


getId(): int
setId(int): void
getName(): String
setName(String): void
getPrice(): double
setPrice(double): void
isExpensive(): boolean
getConnectivity(): String
setConnectivity(String): void
Method[] methods = clazz.getMethods();
for(Method m : methods) {
}


getId(): int
setId(int): void
getName(): String
setName(String): void
getPrice(): double
setPrice(double): void
isExpensive(): boolean
getConnectivity(): String
setConnectivity(String): void
Method[] methods = clazz.getMethods();
getId()
for(Method m : methods) {
if(m.getName().startsWith("get")) {
try {
Object ret = m.invoke(p);
} catch (Exception ex) {
ex.printStackTrace();
}
}
}
p.getId()
getId.invoke(p)